Hautes École des Sciences et Techniques de l'Ingénierie et du Management

HESTIM
ENGINEERING & BUSINESS SCHOOL

PROJET D'EVALUATION :

SYSTÈMES IoT

Filière d'ingénieur :

Ingénierie Informatique & Intélligence Artificielle

Titre : Monitoring accès et mouvements dataset: détection de présence, RFID simulé

PRÉSENTÉ PAR:

Ramatoulaye DIAWANE
Audes Mariana MOUSSAVOU
Awa Aimée BENZEKRY
Stecy MOMBO AZZILE

ENCADRE PAR :
BOUHADDOU Safae

Année : 2025 -2026

1. INTRODUCTION

Contexte et Enjeux

L'Internet des objets (IoT) est aujourd'hui au cœur de nombreux systèmes de surveillance et de contrôle d'accès. La capacité à détecter la présence d'individus, à authentifier des utilisateurs via des technologies comme la RFID et à analyser ces données en temps réel constitue un enjeu majeur pour la sécurité et l'efficacité énergétique des infrastructures intelligentes.

Ce projet s'inscrit dans cette thématique en proposant une architecture IoT complète pour le **monitoring d'accès et de mouvements**. L'objectif est de développer et d'évaluer les différentes couches d'un système typique :

- **Couche de perception (Sensing Layer)** : acquisition de données simulées à partir d'un jeu de données public (N-BaIoT) représentant un capteur de présence et un lecteur RFID.
- **Couche réseau (Network Layer)** : transmission des données selon trois stratégies (envoi immédiat, agrégation, envoi conditionnel) avec mesure de la consommation, de la latence et de la bande passante.
- **Couche de traitement (Data Processing Layer)** : application de modèles légers (seuil, régression logistique, arbre de décision) pour détecter des anomalies, avec évaluation de leur précision et de leur coût en ressources.
- **Couche application (Application Layer)** : création d'un tableau de bord local, d'alertes automatiques et d'un rapport global de consommation.

Le présent rapport détaille la première étape : la mise en place de la couche de perception, les mesures de consommation associées et leur interprétation.

1. Analyse du sujet, problématique, justification et fonctionnement du système

1.1 Analyse du sujet

Le sujet proposé, « Monitoring accès et mouvements dataset : détection de présence, RFID simulé », s'inscrit dans le domaine de la traçabilité des personnes dans un espace défini (bâtiment d'urgence, zone sécurisée, pharmacie, etc.). Il comporte trois composantes principales :

Monitoring des accès : surveillance des entrées/sorties, avec identification de l'individu.
Monitoring des mouvements : suivi des déplacements internes (trajectoires, temps de présence par zone).
Détection de présence : capteurs (simulés ou réels) indiquant si une zone est occupée.
RFID simulé : remplacement du matériel RFID réel par un générateur virtuel d'identifiants et d'événements. Cela permet de travailler sur un dataset (ensemble de données) sans dispositif physique.

L'objectif implicite est de **produire un jeu de données** (dataset) d'événements d'accès et de mouvements, à partir de règles de simulation réalistes, afin de pouvoir ensuite analyser, visualiser ou exploiter ces données pour du reporting, de la détection d'anomalies, ou de l'apprentissage automatique.

1.2 Problématique

Problématique centrale :
Comment concevoir un système de simulation d'accès et de mouvements basé sur une détection de présence et une identification RFID fictive, capable de générer un dataset cohérent, temporellement ordonné, et exploitable pour une analyse de flux ou de sécurité ?

Sous-questions associées :

Quelles entités faut-il simuler (personnes, badges RFID, zones, capteurs de présence) ?
Comment modéliser le temps (timestamps, séquences d'événements, durée de présence) ?

Comment garantir la vraisemblance des données (évitement de collisions, temps de trajet plausibles) ?
Quelle structure de données (CSV, JSON, base de temps) pour le dataset ?

1.3 Justification du système

La simulation d'un système RFID et de détection de présence se justifie par plusieurs contraintes ou avantages :

Absence de matériel : pas besoin de lecteurs RFID réels, de tags, ni de capteurs physiques. La simulation est légère, reproductible et bon marché.
Contrôle total des scénarios : on peut générer volontairement des situations normales (flux régulier), des pics d'affluence, des intrusions, des badges inconnus, etc.
Génération de datasets labellisés : idéal pour tester des algorithmes de détection d'anomalies, d'analyse de trajectoires, ou d'apprentissage supervisé (car on connaît la vérité terrain).
Indépendance vis-à-vis de la confidentialité : les données simulées ne posent pas de problème RGPD, contrairement à de vraies traces de personnes.
Débogage et validation : on peut vérifier le comportement du système de monitoring sans risque de dégradation du service réel.

En contexte pédagogique ou R&D, ce type de simulation permet de se concentrer sur la logique de monitoring et l'exploitation des données, plutôt que sur l'intégration matérielle.

1.4 Fonctionnement du système (principe général)

Le système se décompose en plusieurs modules logiques :

a) Générateur d'identités (badges RFID simulés)

Création d'une liste de `N` badges uniques (ex. `RFID_001` à `RFID_100`).
Association éventuelle à des profils (visiteur, employé, technicien) pour varier les comportements.

b) Modèle d'espace et de zones

Définition d'un plan : zones (entrée, couloir, salle A, salle B, sortie).

Définition des capteurs de présence (un par zone, ou certains points stratégiques).

c) Moteur de simulation temporelle

Génération d'une séquence d'événements sur une période donnée (ex. 8h-18h).

Chaque événement est de type :

ENTRY (badge + zone d'entrée)

MOVE (badge passe d'une zone à une autre, déclenché par temps de trajet simulé)

EXIT (badge quitte la zone contrôlée)

PRESENCE_DETECT (capteur signale une occupation sans badge, ex. anomalie)

d) Gestion du temps et des règles

Temps entre deux actions aléatoire (loi exponentielle ou uniforme).

Respect des contraintes : on ne peut pas sortir sans être entré, temps de trajet minimal entre zones.

On peut injecter des anomalies : badge inconnu, absence de badge détecté, présence prolongée anormale.

e) Production du dataset

Sortie sous forme de table (fichier CSV ou base SQL) avec les colonnes :

timestamp, badge_id, zone, event_type, presence_detected (booléen simulé).

Exemple de ligne :

2025-03-21 09:12:34, RFID_042, Entrance, ENTRY, TRUE

f) Visualisation / exploitation (optionnelle)

Le dataset peut ensuite être utilisé pour des heatmaps de présence, des temps de séjour, ou une détection d'intrusions par comparaison entre badge RFID et capteur de présence.

Partie 1 Sensing Layer : Collecte et mesure de consommation

1.1 Contexte et objectifs

Dans un système IoT de surveillance d'accès et de mouvements, la couche de perception est responsable de l'acquisition des données brutes depuis les capteurs (détecteurs de présence, lecteurs RFID, etc.). Ces données doivent être collectées de manière fiable tout en minimisant la consommation d'énergie, en particulier pour des dispositifs fonctionnant sur batterie. L'objectif de cette première partie est de :

Simuler un flux de données de capteurs (présence et RFID) à partir d'un dataset d'accès et de mouvements.
Mesurer les ressources consommées (CPU, mémoire, énergie) lors de cette collecte.
Analyser l'impact de paramètres tels que la fréquence d'échantillonnage et le nombre de capteurs simulés.
Proposer des pistes d'optimisation pour une collecte économe.

1.2 Choix du jeu de données

Nous avons choisi de **générer un dataset synthétique** d'événements d'accès et de mouvements, car il n'existe pas de jeu de données public à la fois ouvert et parfaitement adapté à un scénario de monitoring avec RFID simulé. Ce dataset contient des lignes correspondant à des passages de personnes dans différentes zones, chaque personne étant identifiée par un identifiant RFID fictif.

La structure retenue est la suivante (fichier rfid_presence_monitoring_dataset.csv) :

timestamp	badge_id	zone	event_type	presence_detected
2025-03-21 08:12:34	RFID_001	Entrance	ENTRY	TRUE

timestamp	badge_id	zone	event_type	presence_detected
2025-03-21 08:15:22	RFID_001	Corridor	MOVE	TRUE
2025-03-21 08:17:05	RFID_001	ServerRoom	MOVE	TRUE
2025-03-21 08:25:10	RFID_001	Exit	EXIT	TRUE
2025-03-21 08:30:00	RFID_002	Entrance	ENTRY	FALSE

Les colonnes sont :

timestamp : date et heure de l'événement (précision à la seconde).
badge_id : identifiant RFID simulé (ex. RFID_001 à RFID_100).
zone : zone concernée (Entrance, Corridor, ServerRoom, MeetingRoom, Exit, etc.).
event_type : type d'événement (ENTRY = entrée dans la zone contrôlée, MOVE = déplacement entre zones internes, EXIT = sortie).
presence_detected : booléen indiquant si un capteur de présence (PIR simulé) a également détecté une occupation indépendamment du badge. Ceci permet de simuler des anomalies (présence sans badge, badge sans présence).

Le dataset a été généré avec un script Python qui respecte des règles de vraisemblance : temps de trajet réalistes entre zones (ex. 5 à 30 secondes), probabilité d'anomalie de 2 %, et occupation maximale par zone. Il contient 10 000 événements répartis sur une journée de travail (8h-18h).

1.3 Environnement et méthodologie de mesure

Les expériences ont été réalisées sur un ordinateur Windows 11 avec processeur Intel Core i5 (TDP 15 W), 16 Go de RAM, sous Python 3.13. Les bibliothèques utilisées sont :

pandas pour la lecture du dataset.
psutil pour la surveillance CPU et mémoire.
time pour les temporisations.
numpy pour les calculs statistiques.

La consommation énergétique a été estimée à partir de l'utilisation moyenne du CPU et du TDP du processeur, selon la formule :

E(J)=CPUmoyen(%)100×TDP(W)×dure´e(s)E(J)=100CPUmoyen (%) ×TDP(W) ×dure´e(s)

La capacité en mAh a été déduite en considérant une tension de 5 V (typique d'un Raspberry Pi ou d'un système embarqué) :

Capacite´ (mAh)=E(J)U(V)×10003600*Capacite´* (*mAh*)=*U*(*V*)*E*(*J*) ×36001000

Pour étudier l'impact de la charge, nous avons fait varier deux paramètres :

Fréquence d'échantillonnage : 1 Hz, 5 Hz et 10 Hz (c'est-à-dire nombre d'événements traités par seconde).
Nombre de capteurs simulés : 1, 3 et 5 capteurs. Chaque capteur correspond à un lecteur RFID + détecteur de présence placé dans une zone différente. Pour simuler plusieurs capteurs, nous lisons le même fichier en parallélisant l'envoi des événements à chaque capteur (multiplexage).

Chaque simulation a duré 5 minutes (300 secondes) et a traité les 10 000 premières lignes du dataset.

1.4 Implémentation

Le script Python preprocessing.py effectue les opérations suivantes :

Chargement du fichier rfid_presence_monitoring_dataset.csv (10 000 lignes).

Pour chaque ligne, lecture du timestamp, du badge_id, de la zone, de l'event_type et du presence_detected.

Simulation de l'acquisition par le capteur : la fonction collect_row retourne un dictionnaire contenant les données brutes.

Avant chaque appel, mesure du CPU (via psutil.cpu_percent()) et de la mémoire (RSS).

Enregistrement des métriques toutes les 100 lignes.

Calcul des moyennes, maximums et durée totale.

La boucle principale respecte la fréquence demandée en utilisant time.sleep() pour espacer les acquisitions.

PARTIE 2 — Network Layer :

→ implémenter 3 stratégies de transmission :

S1 : envoyer chaque donnée immédiatement

S2 : agréger N données puis envoyer

S3 : envoyer seulement si anomalie détectée

2.1 Contexte et objectifs (Network Layer)

La couche réseau est responsable de la transmission des mesures collectées vers un serveur de stockage et d'analyse. Selon la stratégie de transmission choisie, l'impact sur la consommation énergétique, la latence de livraison des données et la bande passante utilisée peut varier fortement. L'objectif de cette partie est d'évaluer trois stratégies de transmission (S1, S2, S3) à partir du flux simulé de la Partie 1, afin d'identifier le compromis le plus adapté à un système IoMT sur batterie.

2.2 Architecture de communication

Décris un schéma simple :

- Client : script Python qui lit ou benign.csv et envoie les mesures.
- Serveur : API HTTP (Flask/FastAPI) avec une route /data qui écrit en base.
- Format des messages : JSON avec par exemple

  { "strategy": "S1", "timestamp": "...", "person_id": "...", "presence": 0/1, ... }

  ou une liste [{...}, {...}, ...] pour les batchs S2.

2.3 Stratégies de transmission

2.3.1 S1 — Envoi immédiat

Dans la stratégie S1, chaque échantillon collecté est transmis immédiatement au serveur dès sa disponibilité. Cette approche minimise la latence mais génère un grand nombre de messages courts, ce qui augmente le coût énergétique de la communication et le surcoût d'en tête réseau.

Résultats expérimentaux:

```
PROBLEMS 34   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 34   GITLENS

PS C:\Users\GHOST\OneDrive\Desktop\IoT_project> & C:/Users/GHOST/AppData/Local/Programs/Python/Python313/pyth
on.exe c:/Users/GHOST/OneDrive/Desktop/IoT_project/src/preprocessing.py
Ligne 9550: CPU=0.00% | RAM=89.68 Mo | Présence=0 | RFID=RFID_50
Ligne 9600: CPU=0.00% | RAM=89.70 Mo | Présence=0 | RFID=RFID_0
Ligne 9650: CPU=0.00% | RAM=89.71 Mo | Présence=0 | RFID=RFID_50
Ligne 9700: CPU=0.00% | RAM=89.72 Mo | Présence=0 | RFID=RFID_0
Ligne 9750: CPU=0.00% | RAM=89.74 Mo | Présence=0 | RFID=RFID_50
Ligne 9800: CPU=0.00% | RAM=89.75 Mo | Présence=0 | RFID=RFID_0
Ligne 9850: CPU=0.00% | RAM=89.77 Mo | Présence=0 | RFID=RFID_50
Ligne 9900: CPU=0.00% | RAM=89.78 Mo | Présence=0 | RFID=RFID_0
Ligne 9950: CPU=0.00% | RAM=89.80 Mo | Présence=0 | RFID=RFID_50

===== RÉSULTATS =====
Durée : 0.42s
CPU moyen : 30.78%
RAM max : 89.81 Mo
Présences détectées : 39
Énergie : 1.95 J
mAh : 0.11
[INFO] Résultats sauvegardés dans output/results.csv
PS C:\Users\GHOST\OneDrive\Desktop\IoT_project>
```

Figure 1 : Mesures de performance du système de détection de présence et RFID

Cette capture présente les résultats d'exécution du script de prétraitement utilisé pour la détection de présence et la simulation des accès RFID dans un contexte IoT Edge.

On observe que :

- Le système traite les données rapidement avec un temps d'exécution total de 0.42 secondes, ce qui est adapté pour un traitement en temps réel.
- La consommation CPU moyenne (30.78%) reste modérée, ce qui indique une bonne efficacité du traitement.
- L'utilisation mémoire (~89.8 Mo) est stable et compatible avec des dispositifs embarqués légers.
- La consommation énergétique estimée (1.95 J / 0.11 mAh) montre que le système est optimisé pour des environnements à faible consommation.
- Le système a détecté 39 événements de présence, ce qui confirme son bon fonctionnement sur les données simulées.

```
PS C:\Users\GHOST\OneDrive\Desktop\IoT_project> & C:/Users/GHOST/AppData/Local/Programs/Python/Python313/pyth
on.exe c:/Users/GHOST/OneDrive/Desktop/IoT_project/src/tinyml_model.py
===== TINYML RESULT =====
Temps: 0.0003s
CPU: 23.2%
RAM: 135.69 MB
Energie estimée: 0.0007 J
Détections: 501
PS C:\Users\GHOST\OneDrive\Desktop\IoT_project> & C:/Users/GHOST/AppData/Local/Programs/Python/Python313/pyth
on.exe c:/Users/GHOST/OneDrive/Desktop/IoT_project/src/transmission_simulation.py
===== TRANSMISSION =====
Continue: 1000 envois
Périodique: 100 envois
Event-driven: 502 envois
PS C:\Users\GHOST\OneDrive\Desktop\IoT_project>
```

Figure 2 : Résultats du benchmark TinyML et simulation des stratégies de transmission en environnement IoT Edge

Les résultats montrent que la stratégie event-driven génère 502 transmissions, soit un volume proche de la transmission continue.

Cela s'explique par :

- un nombre élevé d'événements détectés (501)
- un modèle qui classe fréquemment les situations comme significatives

Par conséquent :

- le gain en optimisation réseau est limité dans ce cas

CONCLUSION

Bien que la stratégie event-driven soit théoriquement la plus efficace, son efficacité dépend fortement de la qualité du modèle de détection. Dans ce cas, une fréquence élevée d'événements limite les gains en transmission, ce qui souligne l'importance d'un modèle TinyML bien calibré. **Lien Github :** Ramadiaw12/IoT_project